

Contents

Projection-Quality Improvement Plan (v1)	1
1. The problem, stated precisely	1
2. Tool inventory	2
2.A — Source-side tools (attack S1)	2
2.B — Projection-side tools (attack S2)	3
2.C — Mesh-side tools (attack S3)	3
3. Tool-to-error mapping	4
4. Phased plan	5
Phase Q-1 — Quantify the error properly (prerequisite)	5
Phase Q-2 — Mesh refinement (attack S3)	5
Phase Q-3 — DG L^2 material projection (attack S2)	5
Phase Q-4 — UCVM native query (attack S1)	6
Phase Q-5 — Convergence study + ship	6
5. Risk register	6
6. Recommendation	7
7. Open questions for the user	7

Projection-Quality Improvement Plan (v1)

Date: 2026-05-09 **Status:** Survey + phased plan — read this before any implementation. **Companion docs:** - data_projection_feature_plan_v2.md (the projection feature these tools improve) - data_projection_schema_v1.md (sidecar schema; unchanged) - system_dev/dynamic_rupture_plan_v4.md (the DG wave operator that consumes projected material)

1. The problem, stated precisely

When the CVM-H sidecar is projected onto the 2000 m mesh and compared to the sidecar’s own value at every tet centroid, the volume-weighted L^2 discrepancy is **~10 % relative** (mean), with $L^\infty \sim \mathbf{2.5\ km/s}$ (max absolute) concentrated at the **basin edges** (LA, Salton, San Andreas trough). The diff plot shows the projection cannot reproduce sub-element basin geometry on this mesh.

The discrepancy decomposes into three independent sources:

$$\begin{aligned} \text{total_error} &= \text{source_error} && (S1) \\ &+ \text{projection_error} && (S2) \\ &+ \text{mesh_resolution_error} && (S3) \end{aligned}$$

Source	What it is	Lever
S1 source	Raw CVM-H is sampled at 0.034° (~3.7 km) lon/lat; the sidecar is itself a trilinear resample to a 1.5 km UTM grid.	Query CVM-H natively (UCVM) instead of the rectilinear sidecar.
S2 projection	ProjectCoefficient point-samples the sidecar at H1-P1 vertices, then linear-interpolates within each tet.	Use a richer projection: L^2 , DG, higher-order FE, or volume-averaging.

Source	What it is	Lever
S3 mesh	The mesh has 581 k vertices total but only ~640 land in the ± 100 m band around $z = -1$ km. Sharp 0.5–3 km/s jumps over a few km can't be resolved by tets that are 5–10 km wide in that band.	Refine the mesh: near-surface, gradient-driven (AMR), or pre-refined from the velocity model.

Key insight from the wave-operator side: the WaveOperator uses L2_FECollection (DG basis, see dynamic/wave_operator.hpp:608). The current driver projects material onto H1 P1 — **inconsistent** with the wave-operator's natural representation. Switching the material representation to **DG L2 of the same order as the wave field** is the most physically aligned fix and is also the one that best preserves basin discontinuities (DG allows jumps at element boundaries).

2. Tool inventory

Three groups, ordered by where they intervene in the pipeline.

2.A — Source-side tools (attack S1)

#	Tool	What it provides	Cost
A1	UCVM (Unified Community Velocity Model)	C/Fortran library; native query of CVM-H at any (lon, lat, depth). Uses CVM-H's internal spline interpolation, higher fidelity than our rectilinear resample . Outputs V_p , V_s , ρ , Q_p , Q_s .	Build dependency (UCVM + CVM-H source data, ~1 GB). One-time install on Frontera & laptop.
A2	In-process libcvmh	UCVM's CVM-H plugin compiled standalone. Avoids UCVM's dispatch layer; smaller deployment.	Same data dependency as A1.
A3	Sub-grid sidecar	Build the sidecar at finer resolution (e.g. 500 m UTM instead of 1500 m). Source data is already at $0.034^\circ \approx 3.7$ km, so sub-3.7 km sidecar resolution is mostly an interpolation refinement.	9× sidecar memory (40 MB $\rightarrow$ 360 MB). Trivial, but doesn't recover information that isn't in the source.
A4	Multi-scale source: combine CVM-H (regional) with high-resolution near-surface basin models (e.g. La Habra basin model, USGS surface-velocity grids)	Best-of-both: CVM-H for far-field, basin models for sharp top-1km features.	New schema; multiple source files; coordinate-of-precedence rules.

2.B — Projection-side tools (attack S2)

#	Tool	What it provides	Cost
B1	L²-projection onto H1 P1	Solve $M \cdot \text{dof} = \int \text{coef} \cdot \varphi dV$. Preserves the integral mean over each element.	One-time mass-matrix solve at startup; ~MFEM LinearForm + BilinearForm + a CG solve. Easy.
B2	DG L²(p)-projection (L2_FECollection(faces; matches the wave-operator's own basis. dim))	Per-element discontinuous polynomials of order p. Naturally allows basin-edge discontinuities at tet faces; matches the wave-operator's own basis.	Per-element projection (no mass solve needed for nodal L ² basis). The wave operator must be extended to read material from a DG GF (Phase 5 of the parent plan, deferred).
B3	Higher-order H1 P2/P3	Quadratic/cubic basis within each tet. Captures sub-tet curvature.	10x/35x DOFs per element. Wave operator must support order > 1. Helps for smooth fields, doesn't help for discontinuities.
B4	Volume-averaging at projection	At each DOF, evaluate the sidecar at K sub-grid points within the supporting elements, average. Roughly equivalent to a low-pass filter at the mesh scale.	K extra interpolations per DOF; one-time. Easy.
B5	GSLIB FindPoints + high-order transfer (mfem::FindPointsGSLIB)	High-quality scattered-point evaluation primitive that MFEM uses for non-conforming transfers. Faster and more accurate than scipy's LinearNDInterpolator.	Already linked in MFEM builds with MFEM_USE_GSLIB.

2.C — Mesh-side tools (attack S3)

#	Tool	What it provides	Cost
C1	Near-surface refinement field in safes_fault_box_nwcut.geo	Add Field[Threshold] keyed off distance to z = 0; force LC_NEAR within e.g. 5 km of the surface.	gmsh edit only. Cheap. Larger mesh in the upper crust.

#	Tool	What it provides	Cost
C2	Gradient-driven AMR	After an initial projection, refine tets where $\ \nabla V_s\ $ exceeds a threshold; re-project; iterate.	Needs MFEM AMR support (already present for tetrahedral). Iterates over: project $\rightarrow$ estimate $\rightarrow$ refine.
C3	Sidecar-driven pre-refinement	Sample sidecar gradient on a fine UTM grid; convert high-gradient cells to a gmsh BackgroundField. The mesher then automatically uses small tets in basin edges.	One-time pre-pass; produces a single mesh. No iteration.
C4	Basin-conformal mesh	Insert basin boundaries (V_s iso-surface) as internal embedded surfaces into the gmsh .geo, so tet faces align with the basin edge.	Requires extracting iso-surfaces from the sidecar and feeding them into gmsh as Surface{ }. More elaborate but eliminates the projection-of-discontinuity problem entirely.
C5	p-adaptivity	Higher polynomial order in cells with high V_s variance.	Substantial code change; MFEM hp infrastructure. Probably overkill.

3. Tool-to-error mapping

Direct rule of thumb: **each tool reduces ONE component, not all three.**

	reduces S1	reduces S2	reduces S3
A1 UCVM	✓✓	—	—
A2 libcvmh	✓✓	—	—
A3 sub-grid sidecar	✓ (small)	—	—
A4 multi-scale src	✓✓	—	—
B1 L^2 -projection	—	✓	—
B2 DG L^2	—	✓✓	(✓ at faces)
B3 higher-order H1	—	✓ (smooth)	—
B4 volume-averaging	—	✓	—
B5 GSLIB FindPoints	(slight)	✓ (precision)	—
C1 near-surface ref	—	—	✓✓
C2 gradient AMR	—	—	✓✓
C3 sidecar-driven pre-ref	—	—	✓✓
C4 basin-conformal mesh	—	(✓ if also B2)	✓✓✓
C5 p-adaptivity	—	✓	—

The **dominant** source of the visible diff in the screenshot is **S3 (mesh resolution at depth)**. S1 and S2 are secondary. An honest order of operations:

1. **First** reduce S3 (mesh refinement) — this is what the visualization exposes most.
 2. **Second** switch to DG L^2 (B2) — this aligns the material representation with the wave operator and is required if you want sharp basin edges to survive at tet faces.
 3. **Third** consider UCVm (A1) only if S1 starts to dominate — i.e. only after S2 and S3 are below a user-acceptable threshold.
-

4. Phased plan

Phase Q-1 — Quantify the error properly (prerequisite)

Goal: unblocked, repeatable error metric so subsequent phases have a target.

Files: - New: `safs/<project>/code_preprocess/data_projection/projection_error.py` - New: `safs/<project>/code_preprocess/data_projection/test_projection_error.py`

Requirements: - Function `compute_error(mesh_vtu_path, sidecar_h5_path, field_name, *, mass_matrix=False, depth_bands=None)` → dict returning M2 (centroid residual), M3 (volume-weighted L^2 and L^∞), and per-band stratification. - Memory-bounded: streamed reading, chunked interpolation, cap at **1 GB peak** so it runs alongside other processes. - One-line CLI wrapper that dumps the metrics as JSON and (optionally) writes a per-tet error VTU for ParaView inspection. - **Tests:** synthetic linear field $f(x,y,z)=ax+by+cz+d$ → error must be $\sim$ machine-precision; constant field → exactly 0.

Acceptance: runs to completion on the 2000 m / 1000 m / 500 m meshes, produces a single JSON with all three metrics per resolution.

Cost: small; pure Python.

Phase Q-2 — Mesh refinement (attack S3)

Goal: drive the L^2 volume-weighted error below $\sim 3\%$ across the upper crust.

Files: - Modified: `safs/<project>/code_meshing/safs_fault_box_nwcut.geo` — add `Field[Threshold]` for z-distance. - Modified: `safs/<project>/code_meshing/run_nwcut_meshing.py` — pass `lc_near_surf`, `dist_surf_inner`, `dist_surf_outer` overrides.

Requirements: - New gmsh `Field[Distance]` measuring $|z|$ (distance from $z = 0$) combined via `Field[Min]` with the existing fault distance field. - Default `LC_NEAR_SURF = LC_NEAR (1500 m)` within `DIST_SURF_INNER = 3000 m` of the surface, decaying to `LC_FAR` over `DIST_SURF_OUTER (40 km)`. - Optional **C3**: build a `Field[PostView]` background size map from the sidecar's V_s gradient; tets shrink where $\|\nabla V_s\|$ exceeds a threshold. - Re-mesh all three resolutions; rerun Phase Q-1 metric.

Acceptance: at the 2000 m mesh, the L^2 error drops by **at least 3x** in the $[-5 \text{ km}, 0]$ depth band (currently $\sim 10\%$, target $\leq 3\%$).

Phase Q-3 — DG L^2 material projection (attack S2)

Goal: preserve basin-edge discontinuities at tet faces; align material representation with the wave operator's basis.

Files: - Modified: `miniapps/seas/io/field_coefficient.{hpp,cpp}` — add `ProjectVelocityDG(sidecar_path, fes_l2)` that takes a `ParFiniteElementSpace` built on `L2_FECollection`. - Modified: `miniapps/seas/dynamic/heterogeneous_material.hpp` — add a `Mode::DG_L2_GridFunction` mode and per-element DOF accessor. - Modified: `miniapps/seas/dynamic/wave_operator.hpp` — overload the ctor to accept the new mode (Phase 5 of the parent plan). - Modified: `miniapps/seas/drivers/project_velocity` — `--basis dg{0,1}` flag.

Requirements: - Default DG order **0** (element-wise constant): one $V_s/V_p/\rho$ value per tet, computed from a centroid sample (or sidecar volume average over the tet via a cubature rule). Matches the standard “constant per element” assumption in earthquake-rupture codes (SeisSol, EXSEIS). - Optional DG order **1** (per-vertex per-element, no continuity constraint at faces): captures linear variation within each tet, jumps allowed across faces. - The post-projection sanity bound check (R-001 fix) is preserved.

Acceptance: - DG-0 projection is byte-equivalent to “centroid sample of the source” to within float precision. - For the basin-edge test region, the L^2 error of DG-0 against the sidecar at element centroids is **lower** than the H1-P1 result (because the H1-P1 result also smooths through nodal averaging within tets). - ParaView output looks visually closer to the sidecar at basin edges.

Phase Q-4 — UCVm native query (attack S1)

Goal: eliminate the trilinear-resample step entirely; query CVM-H at every mesh DOF natively.

Files: - New: `miniapps/seas/io/ucvm_field.hpp` — wraps UCVm’s C API and exposes `amfem::Coefficient` subclass. - Modified: build system — `--with-ucvm-prefix` and library link. - Modified: `safs/<project>/code_preprocess` — gate UCVm behind a feature flag; fall back to the sidecar when UCVm is unavailable.

Requirements: - A new `UCVMCoefficient(model_id, field_name)` that takes (lon, lat, depth) at evaluation, calls UCVm, returns m/s or kg/m^3 . - The driver becomes `--source {sidecar.h5 | ucvm}` and `ProjectVelocity` dispatches accordingly.

Acceptance: - UCVm build succeeds on Frontera (matching the project’s module list in `feedback_sbatch_modules.md`). - Projection of UCVm directly matches the SCEC CVM Explorer output to within 0.1 % at 1000 random query points. - The sidecar path still works (no regression).

Cost: largest of the four phases. Defer until Q-2 + Q-3 are shown insufficient.

Phase Q-5 — Convergence study + ship

Goal: publish the (mesh resolution $\times$ basis order) error chart so users can pick the sweet spot for their compute budget.

Files: - New: `safs/<project>/document/projection_quality_results.md` — table of (mesh-h, basis, L^2 error) and a few representative figures.

Requirements: - Run Phase Q-1 metric on $\{500, 1000, 2000\} \times \{H1-P1, DG-0, DG-1\}$ combinations. - Plot error vs h on a log-log axis (expected $\sim h^1$ for H1-P1 on basin-edge fields; better for DG-1).

Acceptance: a single chart that the user can read to pick the right (mesh, basis) for their target accuracy.

5. Risk register

Risk	Phase	Mitigation
Wave-operator changes to support DG-mode material break BP5/TPV102/TPV104 baselines	Q-3	Phase 5 plan’s regression tests T-5-4..T-5-6 (deferred earlier) must be re-enabled before merging. Constant material is a special case of DG-0 with all elements equal $\rightarrow$ byte-identical baseline.
UCVM build on Frontera fails or conflicts with ICX/IMPI toolchain	Q-4	Build UCVm with the same compiler module list (<code>reference_seisoll_frontera.md</code>). Otherwise fall back to sidecar.

Risk	Phase	Mitigation
Near-surface mesh refinement explodes mesh size (5x–10x tets)	Q-2	Use a non-uniform LC_NEAR_SURF that's only tight in basin regions (sidecar-driven background field, C3).
Memory pressure during error-metric computation	Q-1	Stream-and-chunk implementation with a hard cap; the recent OOM was caused by an unrelated 3.6 GB process.
Q-3 (DG-0) regression on cells with steep gradients (sub-tet basin edges)	Q-3	DG-0 is element-wise constant — represents a step at every face. Combined with C4 (basin-conformal mesh) it's exact.

6. Recommendation

Do Phase Q-1 immediately (quantify what you have). Then **Q-2** (mesh refinement) — that's the largest lever for the diff the user sees in the screenshot. Defer Q-3 / Q-4 until Q-2's metrics motivate them.

Do not start with B1/B2/B3 (projection-side fixes) before Q-2. With the current mesh, those alone won't move the diff plot substantially because the mesh itself can't represent sub-tet basin edges, regardless of how cleverly the DOF values are computed.

7. Open questions for the user

1. What is the **target accuracy**? $\leq 5\% L^2$ for the upper 5 km, or tighter? The answer drives whether Q-2 alone suffices or Q-3 / Q-4 are needed.
2. Is **UCVM permission / data licensing** something you already have set up? CVM-H source needs to be downloaded separately (~1 GB). If not, A3/A4 (sub-grid / multi-scale sidecars) become the only S1 levers.
3. Is there a **specific feature** (e.g., the LA basin) that must be resolved at sub-1 km, or is this a general "smoother diff everywhere" goal? Targeted refinement (C3) is much cheaper than uniform refinement.
4. Should the wave-operator switch to DG- $L^2(0)$ **as the default** for heterogeneous-material runs, with H1-P1 reserved for visualization? This is a design choice with downstream implications.

Send answers and I'll write Phase Q-1 + Q-2 implementation plans (detailed, file-level, with phased acceptance criteria), then start implementation.